

Sotiris Oikonomou

17/04/2026

Assignment 2: Data Stream Processing

Task 1. Setting up Apache Pulsar

Apache Pulsar was set up inside a Docker container running on a medium-flavour Ubuntu 22.04 VM on the Swedish Science Cloud. The Docker approach was chosen over the bare-VM installation because it requires no manual dependency resolution — the official image ships with everything Pulsar needs, and the entire setup reduces to a single command.

```
docker run -it -p 6650:6650 -p 8080:8080 \
  --mount source=pulsardata,target=/pulsar/data \
  --mount source=pulsarconf,target=/pulsar/conf \
  apache/pulsar:2.9.4 bin/pulsar standalone
```

The `-it` flags keep the container's stdout attached to the terminal so the log stream is visible directly. Port **6650** is Pulsar's binary protocol port — the one producers and consumers connect to. Port **8080** is the Admin REST API, used for cluster introspection. The two `--mount` directives attach named Docker volumes for message data and configuration, ensuring both survive a container restart. The final argument, `bin/pulsar standalone`, starts Pulsar in **standalone mode**, which bundles the broker, BookKeeper storage layer, and ZooKeeper coordination service into a single process — appropriate for a single-node lab environment.

Pulsar is confirmed running when the following block appears in the log output:

```
"c-standalone-fw-localhost-8080" : {
  "originalNumAssignments" : 0,
  "finalNumAssignments" : 0,
  "instancesAdded" : 0,
  "instancesRemoved" : 0,
  "instancesUpdated" : 0,
  "alive" : true
}
```

```
second@ubuntu@ec2-130-238-27-193: ~$ ssh -i ~/.ssh/de2-course-key ubuntu@130.238.27.193
Last login: Fri Apr 17 15:52:53 on tty001
second@Mac ~$ ssh -i ~/.ssh/de2-course-key ubuntu@130.238.27.193
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-91-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://lifecycle.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Fri Apr 17 14:13:43 UTC 2026

System load: 0.15          Processes: 115
Usage of /: 28.7% of 19.2GB Users logged in: 1
Memory usage: 48%          IPv4 address for ens3: 192.168.2.252
Swap usage:  0%

Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

*** System restart required ***
Last login: Fri Apr 17 13:48:46 2026 from 90.133.152.94
ubuntu@ec2-130-238-27-193:~$ docker logs $(docker ps -q) | grep -A 10 "alive"
}
}
2026-04-17T13:10:54,447+0000 [pulsar-web-63-8] INFO org.eclipse.jetty.server.RequestLog - 127.0.0.1 - - [17/Apr/2026:13:10:54 +0000] "GET /admin/v2/persistent/public/functions/coordinate/stats/getPreciseBacklog=false&subscriptionBacklogSize=false HTTP/1.1" 200 2081 "-" "Pulsar-Java-v2.9.4" 32
2026-04-17T13:10:54,453+0000 [pulsar-web-63-4] INFO org.eclipse.jetty.server.RequestLog - 127.0.0.1 - - [17/Apr/2026:13:10:54 +0000] "GET /admin/v2/persistent/public/functions/coordinate/stats/getPreciseBacklog=false&subscriptionBacklogSize=false HTTP/1.1" 200 2081 "-" "Pulsar-Java-v2.9.4" 3
2026-04-17T13:11:24,444+0000 [pulsar-web-63-1] INFO org.eclipse.jetty.server.RequestLog - 127.0.0.1 - - [17/Apr/2026:13:11:24 +0000] "GET /admin/v2/persistent/public/functions/coordinate/stats/getPreciseBacklog=false&subscriptionBacklogSize=false HTTP/1.1" 200 2081 "-" "Pulsar-Java-v2.9.4" 23
2026-04-17T13:11:24,468+0000 [pulsar-web-63-5] INFO org.eclipse.jetty.server.RequestLog - 127.0.0.1 - - [17/Apr/2026:13:11:24 +0000] "GET /admin/v2/persistent/public/functions/coordinate/stats/getPreciseBacklog=false&subscriptionBacklogSize=false HTTP/1.1" 200 2081 "-" "Pulsar-Java-v2.9.4" 8
2026-04-17T13:11:24,473+0000 [pulsar-web-63-7] INFO org.eclipse.jetty.server.RequestLog - 127.0.0.1 - - [17/Apr/2026:13:11:24 +0000] "GET /admin/v2/persistent/public/functions/coordinate/stats/getPreciseBacklog=false&subscriptionBacklogSize=false HTTP/1.1" 200 2081 "-" "Pulsar-Java-v2.9.4" 7
2026-04-17T13:11:54,436+0000 [pulsar-web-63-8] INFO org.eclipse.jetty.server.RequestLog - 127.0.0.1 - - [17/Apr/2026:13:11:54 +0000] "GET /admin/v2/persistent/public/functions/coordinate/stats/getPreciseBacklog=false&subscriptionBacklogSize=false HTTP/1.1" 200 2081 "-" "Pulsar-Java-v2.9.4" 7
2026-04-17T13:11:54,444+0000 [pulsar-web-63-4] INFO org.eclipse.jetty.server.RequestLog - 127.0.0.1 - - [17/Apr/2026:13:11:54 +0000] "GET /admin/v2/persistent/public/functions/coordinate/stats/getPreciseBacklog=false&subscriptionBacklogSize=false HTTP/1.1" 200 2081 "-" "Pulsar-Java-v2.9.4" 3
2026-04-17T13:12:24,444+0000 [pulsar-web-63-6] INFO org.eclipse.jetty.server.RequestLog - 127.0.0.1 - - [17/Apr/2026:13:12:24 +0000] "GET /admin/v2/persistent/public/functions/coordinate/stats/getPreciseBacklog=false&subscriptionBacklogSize=false HTTP/1.1" 200 2081 "-" "Pulsar-Java-v2.9.4" 32
ubuntu@ec2-130-238-27-193:~$
```

Task 2. Producing and Consuming Messages

A. Single machine

With Pulsar running, a consumer and producer were implemented in Python using the pulsar-client==2.9.4 library and executed in separate terminal panes on the same VM. The consumer subscribes to DETopic under the subscription name DE-sub and blocks on consumer.receive() until a message arrives. On successful receipt it prints the message payload and acknowledges delivery; on failure it negative-acknowledges, signalling Pulsar to redeliver.

```
# consumer.py
import pulsar

client = pulsar.Client('pulsar://localhost:6650')
consumer = client.subscribe('DETopic', subscription_name='DE-sub')

msg = consumer.receive()
try:
    print("Received message : %s" % msg.data())
    consumer.acknowledge(msg)
except:
    consumer.negative_acknowledge(msg)

client.close()
```

```
# producer.py
import pulsar
```

The consumer was started first so it was already waiting when the producer ran. The message appeared in the consumer terminal immediately on send.

[illegible]

The same setup was then distributed across two VMs. Apache Pulsar and the producer ran on **VM A** (sotirisoikonomou-pulsar-vm, 130.238.27.193); the consumer ran on **VM B** (sotirisoikonomou-consumer-vm, 130.238.27.203). The only change required was updating the client connection string in consumer.py to point at VM A's address:

```
client = pulsar.Client('pulsar://130.238.27.193:6650')
```

The **public/floating IP** was used rather than the private internal address (192.168.*) because VM B routes to VM A over the cloud's external network — the private address is not reachable across machines

The consumer on VM B successfully received the message produced on VM A, confirmed by the output log showing the connection traversing 192.168.2.30:42690 → 130.238.27.193:6650 and printing:

[illegible]

```
sccon - ubuntu@pulsarctlshanno-consumer-m0: ~$ ssh -A -C ssh@pulsar-150.238.27.203 - 280.620
ubuntu@pulsarctlshanno-consumer-m0: ~$ ssh -A -C ssh@pulsar-150.238.27.203
Scanning processes...
Scanning candidates...
Scanning linux images...

Running kernel seems to be up-to-date.

Restarting services...
systemctl restart cron.service libbalance.service multipathd.service packagekit.service polkit.service rsyslog.service serial-getty@tty50.service snapd.service ssh.service systemd-journald.service systemd-networkd.service systemd-resolved.service systemd-timesyncd.service systemd-udevd.service
Service restarts being deferred:
  At: mediastore/restart-debconf.service
systemctl restart getty@tty5.service
systemctl restart networkd-dispatcher.service
systemctl restart systemd-logind.service
systemctl restart unattended-upgrades.service
systemctl restart user@1000.service

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.

ubuntu@pulsarctlshanno-consumer-m0: ~$ ls
ubuntu@pulsarctlshanno-consumer-m0: ~$ sudo apt install python3-pip
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3-pip is already the newest version (22.0.2+deb1ubuntu1.7).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
debconf: warning: 'python3-pip' $ pip install python-client-2.9.4
Defaulting to user installation because normal site-packages is not writable
Collecting pulsar-client-2.9.4
  Downloading pulsar-client-2.9.4-cp310-cp310-linux_x86_64.manylinux2014_x86_64.whl (31.5 MB)
Requirement already satisfied: certifi in /usr/lib/python3/dist-packages (from pulsar-client-2.9.4) (2020.6.20)
Requirement already satisfied: six in /usr/lib/python3/dist-packages (from pulsar-client-2.9.4) (c.16.0)
Installing collected packages: pulsar-client
Successfully installed pulsar-client-2.9.4
ubuntu@pulsarctlshanno-consumer-m0: ~$ nano consumer.py
python3 consumer.py
2020-04-17 14:00:39.877 INFO [19843180770843] Client 18.1 Subscribing on Topic /DEtopic
2020-04-17 14:00:39.878 INFO [1984318078163] ExecutorService-41 I Run a service in a single thread
2020-04-17 14:00:39.878 INFO [19843180770843] ClientConnection:181 [Consumer:/public/default/DEtopic, DE-sub, 0] GetSetting connection from pool
2020-04-17 14:00:39.878 INFO [19843180770843] ConnectionPool:96 [Create connection for pulsar://150.238.27.193:6650
2020-04-17 14:00:39.882 INFO [1984318078163] ClientConnection:125 [192.168.2.38-42706 -> 130.238.27.193:6650] Connected to broker
2020-04-17 14:00:39.890 INFO [1984318078163] Handler-Rx-03 [Persistent:/public/default/DEtopic, DE-sub, 0] GetSetting connection from pool
2020-04-17 14:00:39.890 INFO [19843180770843] ExecutorService-41 I Run a service in a single thread
2020-04-17 14:00:39.892 INFO [1984318078163] ClientConnection:125 [192.168.2.38-42706 -> 130.238.27.193:6650] Connected to broker
2020-04-17 14:00:39.903 INFO [1984318078163] ConnectionPool:96 [Create connection for pulsar://localhost:6650
2020-04-17 14:00:39.904 INFO [1984318078163] ClientConnection:127 [192.168.2.38-42706 -> 130.238.27.193:6650] Connected to broker through proxy. Local broker: pulsar://localhost:6650
2020-04-17 14:00:39.914 INFO [1984318078163] ConsumerImpl:128 [Persistent:/public/default/DEtopic, DE-sub, 0] Created consumer on broker: 192.168.2.38-42706 -> 130.238.27.193:6650
Received message: "Welcome to Data Engineering Course!"
2020-04-17 14:01:00.233 INFO [19843180770843] ClientImpl:506 [Closing Pulsar Client with 0 producers and 0 consumers
2020-04-17 14:01:00.334 INFO [19843180770843] ConsumerImpl:185 [Persistent:/public/default/DEtopic, DE-sub, 0] Closing consumer for topic persistent:/public/default/DEtopic
2020-04-17 14:01:00.347 INFO [1984318078163] ConsumerImpl:195 [Persistent:/public/default/DEtopic, DE-sub, 0] Closed consumer
2020-04-17 14:01:00.348 INFO [19843180770843] ClientConnection:154 [192.168.2.38-42706 -> 130.238.27.193:6650] Connection closed
2020-04-17 14:01:00.348 INFO [19843180770843] ClientConnection:263 [192.168.2.38-42706 -> 130.238.27.193:6650] Destroyed connection
2020-04-17 14:01:00.349 INFO [19843180770843] ClientConnection:251 [192.168.2.38-42706 -> 130.238.27.193:6650] Connection closed
2020-04-17 14:01:00.349 INFO [19843180770843] ClientConnection:263 [192.168.2.38-42706 -> 130.238.27.193:6650] Destroyed connection
2020-04-17 14:01:00.350 INFO [1984318078163] ExecutorService-47 I Event loop of ExecutorService exits successfully
2020-04-17 14:01:00.350 INFO [19843180770843] ConsumerImpl:128 I Event loop of ExecutorService exits successfully
ubuntu@pulsarctlshanno-consumer-m0: ~$
```

Task 3. Conceptual Questions

Question 1: Running Pulsar in Docker is more convenient but comes with trade-offs. The container abstraction adds networking overhead on port 6650 and slightly slower disk I/O through volume mounts — things that matter at high throughput. A native installation avoids all of this since the process talks directly to system resources. That said, Docker wins on portability: the same image runs identically on any machine without manually installing Java, configuring paths, or managing dependencies. For a lab environment this is clearly the right call. For a production system processing millions of messages per second, native or Kubernetes-managed deployments make more sense.

Question 2: Pulsar adds several things Kafka simply does not have out of the box. Native multi-tenancy through a tenant/namespace/topic hierarchy means one cluster can serve multiple teams with isolated permissions and quotas — Kafka has nothing equivalent. Tiered storage lets Pulsar offload older data to S3 or similar without consumer-facing changes, whereas Kafka needs third-party tooling for this. Pulsar Functions give you a built-in lightweight compute layer for simple transformations without spinning up a separate Flink or Spark job. The biggest architectural difference though is that Pulsar separates brokers (compute) from BookKeeper (storage), so you can scale each independently. In Kafka they are coupled, so scaling storage means scaling brokers too.

Question 3: The fundamental problem with batch processing is latency — you cannot get results faster than the batch interval, regardless of how fast your compute is. At scale a second issue appears: batch jobs spike resource consumption during execution and sit idle in between, which is wasteful. Streaming systems like Pulsar fix both problems by processing events as they arrive, spreading load evenly over time. Pulsar specifically stores messages in a persistent distributed log, so consumers can fall behind and catch up without data loss, and multiple independent consumers can read the same stream at different speeds. The literature (Hesse & Lorenz, 2015) frames this as moving from periodic snapshots of state to a continuous, real-time view — which is what modern data-intensive applications actually need.

Question 4: Pulsar is built around the **publish-subscribe pattern**. Producers write to a topic without knowing anything about who is reading; consumers subscribe independently and receive messages based on their subscription type. The main benefit is loose coupling — you can add a new consumer to an existing topic without touching the producer, fan the same stream out to multiple pipelines simultaneously, and handle temporary consumer downtime gracefully since Pulsar holds messages until they are acknowledged. Compared to point-to-point messaging where the sender needs to know the receiver, pub/sub scales much more naturally to complex data flows.

Question 5: Pulsar has four subscription modes. **Exclusive** gives all messages to one consumer only — straightforward, preserves order, but no parallelism. **Shared** distributes messages across multiple consumers in round-robin, which scales throughput but drops ordering guarantees. **Failover** keeps one active consumer with standbys ready to take over on failure, giving you ordering with high availability. **Key_Shared** sends messages with the same key to the same consumer consistently, so you get parallel processing while still maintaining per-key order — useful when you are partitioning by user ID or device. Which mode to use depends entirely on whether ordering or throughput is the priority.

Question 6: ZooKeeper handles the coordination work that a distributed system cannot do without — broker metadata, leader election, and in Pulsar's case, BookKeeper ledger locations too. Without something playing this role, nodes cannot discover each other or agree on who owns what after a failure. Whether ZooKeeper specifically is required is a different question. Kafka answered it by replacing ZooKeeper with its own KRaft consensus protocol in version 3.3. Pulsar is working on a similar replacement called oxia. So the coordination layer is non-negotiable, but ZooKeeper itself is not — it is just the traditional solution that both systems are now moving away from.

Question 7: Pulsar's architecture has four main components. **Brokers** are stateless servers that handle client connections and route messages — because they hold no data, scaling or replacing them requires no migration. **Apache BookKeeper** is the storage layer; message segments are written across multiple Bookie nodes for replication and durability. **ZooKeeper** manages cluster metadata and coordination. On top of this sits **Pulsar Functions and Pulsar IO**, which provide built-in stream processing and connectors to external systems. The key design choice is the separation of brokers from storage, which gives Pulsar more flexibility than systems where compute and storage are bundled together.

Question 8: A broker is the entry point for all client traffic in Pulsar. It accepts producer connections, writes incoming messages to BookKeeper, manages subscription state, and dispatches messages to consumers. The important detail is that brokers are **stateless** — they do not store data themselves. This means a failed broker can be replaced immediately with no data loss, and new brokers can be added to handle more topics without rebalancing stored data. Each topic partition is owned by one broker at a time, but ownership transfers automatically if that broker goes down. This contrasts with Kafka where broker failure means the data on that broker is temporarily unavailable until replication catches up.

Question 9: Tenants are Pulsar's way of supporting multiple independent users on one cluster. Each tenant controls which clusters it can use and which roles have admin access. Under a tenant, namespaces group related topics and carry shared policies — retention rules, replication targets, storage quotas, and access controls. This hierarchy means a university, for example, could run one Pulsar cluster where each department is a tenant with its own namespace and topics, fully isolated from each other. This is something Kafka does not support natively and is one of the main reasons organisations with multi-team data platforms choose Pulsar.

Question 10: Without log compaction, topics that act as state stores become a problem over time. Every update appends a new message, so a topic tracking user preferences or device states accumulates thousands of historical entries per key, most of which are irrelevant. A consumer restarting from the beginning has to replay all of this to get current state, which gets more expensive as the topic grows. Log compaction solves this by keeping only the latest message per key and discarding the rest. Without it, storage grows indefinitely and consumer recovery times increase proportionally with data age — which becomes a serious operational issue at scale.

Question 11: A topic is the logical channel — producers write to it, consumers read from it. By default a topic lives on one broker, so its throughput is capped by what that single broker can handle. A partitioned topic splits the logical channel into multiple independent sub-topics, each managed by a different broker and stored in separate BookKeeper ledgers. Producers distribute messages across partitions by key hash or round-robin, and consumers read from different partitions in parallel. From the application's perspective it

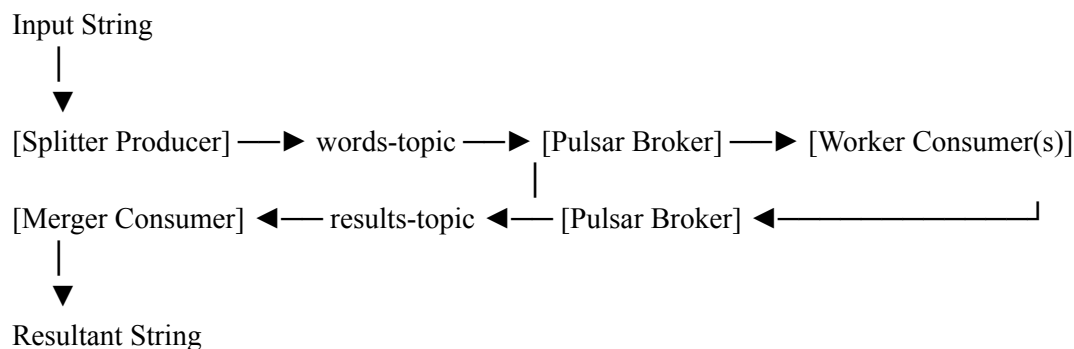
is still one topic; underneath, it is spread across the cluster. Partitioning is simply how Pulsar scales beyond a single broker's limits.

Question 12: When a consumer calls `acknowledge()`, it tells Pulsar the message was processed successfully. Pulsar advances the subscription cursor past that message, so it will not be delivered again. If the consumer crashes before acknowledging, Pulsar redelivers the message to the next available consumer — this is the basis of the **at-least-once delivery guarantee**. Calling `negative_acknowledge()` explicitly signals processing failure and triggers immediate redelivery without waiting for a timeout. The implication is that consumers need to handle duplicate delivery gracefully, since a message could be redelivered if the consumer crashes after processing but before acknowledging. Without this acknowledgement loop, Pulsar has no way to know whether a delivered message was actually handled.

Task 4. Splitting and Merging Operations with Apache Pulsar

The original `conversion.py` processes words sequentially in a single loop. Each word is converted one at a time on a single thread, and the entire input string must be loaded into memory upfront. At millions of words this breaks down in two ways: processing time scales linearly with input size since there is no parallelism, and memory consumption grows proportionally with the string length. There is also no fault tolerance — if the process crashes mid-loop, all progress is lost and the entire job must restart from scratch. The architecture is fundamentally single-machine and single-threaded, which is incompatible with big data workloads.

The redesign distributes the workload across three roles connected through Apache Pulsar:



The **Splitter Producer** reads the input string, splits it into individual words, and publishes each word as a separate message to `words-topic`, tagged with its index and the total word count. The **Worker Consumer** subscribes to `words-topic` using a Shared subscription, applies the conversion function to each word, and publishes the result with its index to `results-topic`. Using a Shared subscription means multiple worker instances can consume from `words-topic` in parallel, which is the key scalability improvement over the original sequential loop. The **Merger Consumer** subscribes to `results-topic`, collects converted words, and reconstructs the final string in the correct order once all words have been received.

```

# splitter.py
import pulsar
import json

INPUT_STRING = "I want to be capatilized"

client = pulsar.Client('pulsar://localhost:6650')
producer = client.create_producer('words-topic')

words = INPUT_STRING.split()
total = len(words)

print(f'Original String: {INPUT_STRING}')
print(f'Splitting {total} words into words-topic...')

for i, word in enumerate(words):
    message = json.dumps({"index": i, "word": word, "total": total})
    producer.send(message.encode('utf-8'))
    print(f'Sent word [{i}]: {word}')

producer.flush()
client.close()
print("All words sent.")

# worker.py
import pulsar
import json

def function(string):
    return string.upper()

client = pulsar.Client('pulsar://localhost:6650')
consumer = client.subscribe('words-topic',
                             subscription_name='worker-sub',
                             consumer_type=pulsar.ConsumerType.Shared)
producer = client.create_producer('results-topic')

print("Worker ready, waiting for words...")

while True:
    msg = consumer.receive()
    try:
        data = json.loads(msg.data().decode('utf-8'))
        converted = function(data['word'])

```



```

        result = json.dumps({"index": data['index'],
                             "converted": converted,
                             "total": data['total']})
        producer.send(result.encode('utf-8'))
        print(f'Processed [{data['index']}: {data['word']} -> {converted}'))
        consumer.acknowledge(msg)
    except Exception as e:
        print(f'Error: {e}')
        consumer.negative_acknowledge(msg)

# merger.py
import pulsar
import json

client = pulsar.Client('pulsar://localhost:6650')
consumer = client.subscribe('results-topic', subscription_name='merger-sub')

print("Merger ready, waiting for results...")

results = {}
total = None

while True:
    msg = consumer.receive()
    try:
        data = json.loads(msg.data().decode('utf-8'))
        results[data['index']] = data['converted']
        total = data['total']
        print(f'Received [{data['index']}: {data['converted']}')
        consumer.acknowledge(msg)

        if len(results) == total:
            final = ''.join(results[i] for i in sorted(results.keys()))
            print(f'\nResultant String: {final}')
            break
    except Exception as e:
        print(f'Error: {e}')
        consumer.negative_acknowledge(msg)

client.close()

```

The three components ran on a single VM using separate tmux panes. The merger was started first to ensure it was subscribed before results arrived, followed by the worker, then the splitter. The output

```

ubuntu@stratoskonnemu-pulsar-v6 -- ssh -l stratoskonnemu-course-jay ubuntu@190.238.27193 -- 280x62
ubuntu@stratoskonnemu-pulsar-v6 ~$ cd /tmp && ls -la
total 4
drwxr-xr-x 2 root root 4096 Aug 20 23:28
ubuntu@stratoskonnemu-pulsar-v6 ~$ tmux split-window -v
tmux split-window -v
ubuntu@stratoskonnemu-pulsar-v6 ~$ python3 merger.py
280x-04-17 17:48:13.282 INFO [INFO] Client#88 | Subscribing on Topic results-topic
280x-04-17 17:48:13.282 INFO [INFO] ExecutorService#41 | Run i.service in a single thread
280x-04-17 17:48:13.283 INFO [INFO] ClientConnection#189 | [Consumer => pulsar://localhost:6500] Create ClientConnection, timeout=10000
280x-04-17 17:48:13.283 INFO [INFO] ConnectionPool#96 | Created connection for pulsar://localhost:6500
280x-04-17 17:48:13.284 INFO [INFO] ClientConnection#175 | [IDZ.0.0.1:54018 -> 127.0.0.1:6500] Connected to broker
280x-04-17 17:48:13.292 INFO [INFO] HandlerBase#61 | [persistent://public/default/results-topic, merger-sub, 0] Getting connection from pool
280x-04-17 17:48:13.292 INFO [INFO] ExecutorService#41 | Run i.service in a single thread
280x-04-17 17:48:13.292 INFO [INFO] ConsumerImpl#218 | [persistent://public/default/results-topic, merger-sub, 0] Created consumer on broker [127.0.0.1:54018 -> 127.0.0.1:6500]
280x-04-17 17:48:13.303 INFO [INFO]
Merger ready, waiting for results...
Received [0]:
Received [1]: WANT
Received [2]: TO
Received [3]: CAPITALIZED
Received [4]: BE
Resultant String: I WANT TO BE CAPITALIZED
280x-04-17 17:48:38.412 INFO [INFO] ClientImpl#500 | Closing Pulsar client with 0 producers and 1 consumers
280x-04-17 17:48:38.412 INFO [INFO] ConsumerImpl#885 | [persistent://public/default/results-topic, merger-sub, 0] Closing consumer for topic persistent://public/default/results-topic
280x-04-17 17:48:38.412 INFO [INFO] ConsumerImpl#935 | [persistent://public/default/results-topic, merger-sub, 0] Closed consumer 0
280x-04-17 17:48:38.412 INFO [INFO] ClientConnection#1563 | [127.0.0.1:54018 -> 127.0.0.1:6500] Connection closed
280x-04-17 17:48:38.412 INFO [INFO] ExecutorService#97 | Event loop of ExecutorService exits successfully
280x-04-17 17:48:38.412 INFO [INFO] ExecutorService#47 | Event loop of ExecutorService exits successfully
280x-04-17 17:48:38.412 INFO [INFO] ClientConnection#263 | [127.0.0.1:54018 -> 127.0.0.1:6500] Destroyed connection
ubuntu@stratoskonnemu-pulsar-v6 ~$

ubuntu@stratoskonnemu-pulsar-v6 ~$ python3 worker.py
280x-04-17 17:48:38.412 INFO [INFO] ClientImpl#500 | Closing Pulsar client with 1 producers and 0 consumers
280x-04-17 17:48:38.412 INFO [INFO] ConsumerImpl#885 | [persistent://public/default/results-topic, merger-sub, 0] Closing consumer for topic persistent://public/default/results-topic
280x-04-17 17:48:38.412 INFO [INFO] ConsumerImpl#935 | [persistent://public/default/results-topic, merger-sub, 0] Closed consumer 0
280x-04-17 17:48:38.412 INFO [INFO] ClientConnection#1563 | [127.0.0.1:54018 -> 127.0.0.1:6500] Connection closed
280x-04-17 17:48:38.412 INFO [INFO] ExecutorService#97 | Event loop of ExecutorService exits successfully
280x-04-17 17:48:38.412 INFO [INFO] ExecutorService#47 | Event loop of ExecutorService exits successfully
280x-04-17 17:48:38.412 INFO [INFO] ClientConnection#263 | [127.0.0.1:54018 -> 127.0.0.1:6500] Destroyed connection
ubuntu@stratoskonnemu-pulsar-v6 ~$

ubuntu@stratoskonnemu-pulsar-v6 ~$ python3 splitter.py
280x-04-17 17:48:38.358 INFO [INFO] ClientConnection#189 | [Consumer=>pulsar://localhost:6500] Create ClientConnection, timeout=10000
280x-04-17 17:48:38.358 INFO [INFO] ConnectionPool#96 | Created connection for pulsar://localhost:6500
280x-04-17 17:48:38.358 INFO [INFO] ClientConnection#175 | [IDZ.0.0.1:55468 -> 127.0.0.1:6500] Connected to broker
280x-04-17 17:48:38.362 INFO [INFO] HandlerBase#61 | [persistent://public/default/words-topic, 0] Getting connection from pool
280x-04-17 17:48:38.367 INFO [INFO] ProducerImpl#173 | [persistent://public/default/words-topic, 0] Created producer on broker [127.0.0.1:55468 -> 127.0.0.1:6500]
Original String: I want to be capitalized
Splitting words into words-topic...
Sent word [0]:
Sent word [1]: want
Sent word [2]: to
Sent word [3]: be
Sent word [4]: capitalized
280x-04-17 17:48:38.388 INFO [INFO] ClientImpl#500 | Closing Pulsar client with 1 producers and 0 consumers
280x-04-17 17:48:38.388 INFO [INFO] ProducerImpl#590 | [persistent://public/default/wds-topic, standalone-1-5] Closing producer for topic persistent://public/default/wds-topic
280x-04-17 17:48:38.389 INFO [INFO] ProducerImpl#630 | [persistent://public/default/wds-topic, standalone-1-5] Closed producer
280x-04-17 17:48:38.389 INFO [INFO] ClientConnection#1551 | [IDZ.0.0.1:55468 -> 127.0.0.1:6500] Connection closed
280x-04-17 17:48:38.389 INFO [INFO] ClientConnection#263 | [127.0.0.1:55468 -> 127.0.0.1:6500] Destroyed connection
280x-04-17 17:48:38.393 INFO [INFO] ExecutorService#47 | Event loop of ExecutorService exits successfully
All words sent
280x-04-17 17:48:38.393 INFO [INFO] ProducerImpl#559 | Producer : [persistent://public/default/wds-topic, standalone-1-5], [C
batching = off]
ubuntu@stratoskonnemu-pulsar-v6 ~$

```